

AACAP26 - OpenMP Challenge

Obiettivi e Regole

Questa challenge vi farà ragionare su vari argomenti che abbiamo trattato nel corso finora; in particolare, se torniamo ai risultati di apprendimento attesi presentati nella lezione introduttiva, allena le vostre capacità di

“sfruttare la conoscenza dell’architettura del calcolatore per migliorare le prestazioni dei programmi”.

Compilazione e consegna:

- Rispondete alle domande aggiungendo pagine in fondo a questo PDF (non cancellate la descrizione del problema).
- Salvate **due copie** del documento con le risposte, una con i nomi dei partecipanti al gruppo in una tabellina simile a quella in fondo a questa pagina e una *blind*.
- Mandate a serena.curzel@polimi.it due file:
 - Il documento PDF con i nomi;
 - Un archivio .zip contenente il documento PDF *blind* e tutto il codice ed eventuali script necessari per riprodurre gli esperimenti, sia nella versione base sia nell’eventuale versione ottimizzata.

Buon lavoro!

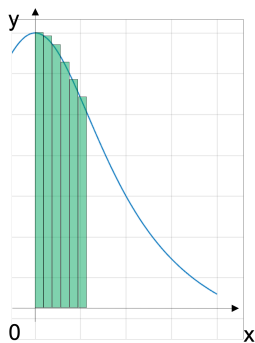
Nome e Cognome	Codice Persona

Descrizione del problema

Uno dei metodi per calcolare il valore di π è la soluzione del seguente integrale:

$$\int_0^1 \frac{4}{1+x^2} dx = \pi$$

Possiamo utilizzare l'integrale di Riemann e approssimare il valore dell'integrale calcolando l'area di tanti piccoli rettangoli:



Ogni rettangolo è indipendente da tutti gli altri e può dunque essere gestito in parallelo. Ecco uno snippet di codice che utilizza OpenMP per implementare il calcolo:

```
#pragma omp parallel
{
    int i;
    int id = omp_get_thread_num();
    int numthreads = omp_get_num_threads();
    double x;
    sum[id] = 0.0;

    for (i=id; i<num_steps; i+=numthreads){
        x = (i+0.5)*step;
        sum[id] = sum[id] + 4.0/(1.0+x*x);
    }
}

for (i=0; i<4; i++)
    full_sum += sum[i];

pi = step * full_sum;
```

Domande

1. **Cosa manca per poter eseguire il programma?** Completate il codice con l'inizializzazione corretta delle variabili, e aggiungete la misurazione del tempo totale di esecuzione.
2. **Il tempo di esecuzione diminuisce all'aumentare del numero di thread utilizzati?** Eseguite dei test con diversi numeri di thread, elencate le caratteristiche della macchina che avete usato per eseguirli (sistema operativo, numero di core, versione di OpenMP), e riportate in una tabella o in un grafico i risultati.
3. **Come mai la diminuzione del tempo di esecuzione non è lineare?** Fate delle ipotesi in base alle vostre conoscenze dell'architettura del calcolatore.
4. **Si può rendere il codice più efficiente?** In base alla risposta che avete trovato per il punto 3, modificate il programma e ripetete gli esperimenti del punto 2 per dimostrare il miglioramento nel tempo di esecuzione.

1 Domanda 1

Cosa manca per poter eseguire il programma? Completate il codice con l'inizializzazione corretta delle variabili, e aggiungete la misurazione del tempo totale di esecuzione.

1.1 Codice del programma

```
// inclusione librerie
#include <omp.h>
#include <stdio.h>
#include <math.h>

int num_threads = 4;
long long num_steps = 10000000;

int main()
{
    // set del numero di thread
    omp_set_num_threads(num_threads);

    // inizializzazione timer globale
    double start_time = omp_get_wtime();

    // dichiarazione variabili
    double step = 1.0/((double) num_steps);
    double sum[num_threads];
    double full_sum = 0.0;
    double pi;
    int j; // indice in piu per il for finale

    // inizializzazione timer parallelo, con solo
    //   ↪ overhead di creazione thread
    double start_parallel_time = omp_get_wtime();

    #pragma omp parallel num_threads(num_threads)
    {
        long long i; // long long per fare test con
        //   ↪ numero di step elevati
        int id = omp_get_thread_num();
        // printf("thread id: %d\n", id);
        int numthreads = omp_get_num_threads();
        double x;
        sum[id] = 0.0;

        for (i=id; i<num_steps; i+=numthreads){
```

```

        x = (i+0.5)*step;
        sum[id] = sum[id] + 4.0/(1.0+x*x);
    }
}

for(j=0; j<num_threads; j++)
    full_sum += sum[j];

pi=step*full_sum;

// calcolo tempo totale e tempo parallelo
double total_time = omp_get_wtime() - start_time;
double total_parallel_time = omp_get_wtime() -
    ↪ start_parallel_time;

// stampe finali
printf(" Approssimazione di pi: -%.16f\nNumero di -
    ↪ threads: -%d\nNumero di passi: -%lld\nTempo -
    ↪ totale di esecuzione: -%.6f\nTempo totale -
    ↪ calcolo parallelo: -%.6f\n", pi, num_threads,
    ↪ num_steps, total_time, total_parallel_time);

return 0;
}

```

1.2 Risultati dei test

Facendo variare il numero di step, otteniamo stime sempre più precise di π .

Approssimazione di pi: 3.1416009869231249

Numero di threads: 4

Numero di passi: 100

Tempo totale di esecuzione: 0.001000

Tempo totale calcolo parallelo: 0.001000

Approssimazione di pi: 3.1415926535899033

Numero di threads: 4

Numero di passi: 1000000

Tempo totale di esecuzione: 0.004000

Tempo totale calcolo parallelo: 0.004000

Approssimazione di pi: 3.1415926535897682

Numero di threads: 4

Numero di passi: 1000000000

Tempo totale di esecuzione: 2.400000
Tempo totale calcolo parallelo: 2.400000

Notiamo che il tempo di esecuzione totale e il tempo di esecuzione della sezione parallela del codice (comprensivo della parte di computazione e dell'overhead dato dalla creazione dei thread) coincidono sempre. Questo perché, probabilmente, il tempo di esecuzione sequenziale per l'inizializzazione delle variabili è praticamente nullo. Per la legge di Amdahl sarebbe dunque in teoria possibile ottenere (su una macchina ideale) uno speedup pressoché lineare a dimensione del problema fissata.

2 Domanda 2

Il tempo di esecuzione diminuisce all'aumentare del numero di thread utilizzati? Eseguite dei test con diversi numeri di thread, elencate le caratteristiche della macchina che avete usato per eseguirli (sistema operativo, numero di core, versione di OpenMP), e riportate in una tabella o in un grafico i risultati.

2.1 Specifiche della Macchina

1. **Sistema Operativo:** Windows 11 Home;
2. **Numero di Core:** 12 core e 16 threads (processore Intel Core i7-1260P);
3. **Versione di OpenMP:** 201307;

2.2 Risultati dei test

L'esecuzione in ciascuno dei casi presenti è stata effettuata sulla stessa macchina in condizioni simili. I risultati ottenuti sono la media di 20 simulazioni (per eliminare parzialmente gli effetti della variabilità dei tempi di esecuzione) e sono mostrati in Tables 1, 2 e Figures 1, 2.

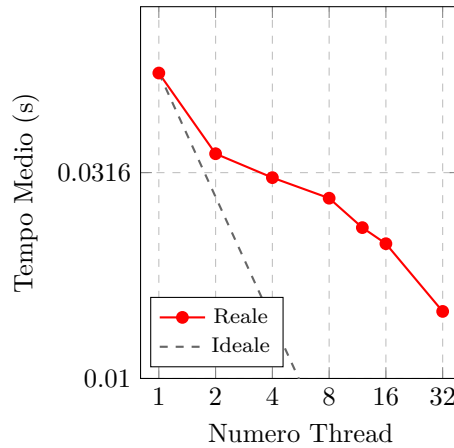
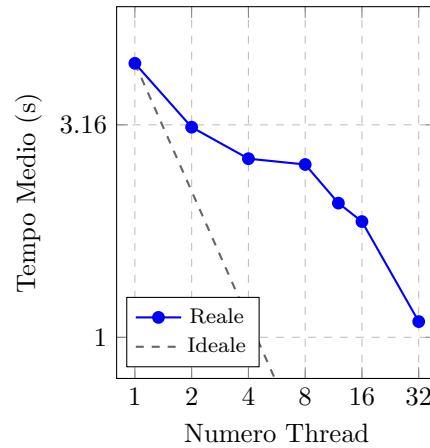
L'esperimento è stato ripetuto con due numeri di step differenti (10^7 , 10^9). Ci saremmo aspettati che, aumentando il numero di step, e quindi il carico parallelizzabile, si ottenesse uno speedup migliore ma ciò apparentemente non è stato confermato dalle evidenze sperimentali. Supponiamo questo sia dovuto a delle leggere differenze nello stato della macchina durante l'esecuzione (ad esempio piccole variazioni dei processi attivi in background al momento dei test).

Table 1: Risultati per 10^7 step

Threads	Tempo (s)	Speedup
1	0.0552	1.00
2	0.03515	1.57
4	0.03075	1.80
8	0.02741	2.01
12	0.02325	2.37
16	0.02125	2.60
32	0.01455	3.79

Table 2: Risultati per 10^9 step

Threads	Tempo (s)	Speedup
1	4.4141	1.00
2	3.1227	1.41
4	2.6326	1.68
8	2.5508	1.73
12	2.06915	2.13
16	1.87245	2.36
32	1.0888	4.05

Tempo vs Numero Thread 10^7 step. Tempo vs Numero Thread 10^9 step.Figure 1: Tempo medio vs Ideale per 10^7 step.Figure 2: Tempo medio vs Ideale per 10^9 step.

3 Domanda 3

Come mai la diminuzione del tempo di esecuzione non è lineare? Fate delle ipotesi in base alle vostre conoscenze dell'architettura del calcolatore.

Esaminiamo prima il caso in cui il numero di thread richiesti è minore del numero di thread fisicamente disponibili sulla macchina.

In questo caso, ci saremmo aspettati un andamento teorico lineare, in quanto, a meno che non ci siano thread utilizzati da un altro programma o dal sistema operativo, ogni nuovo thread creato dal nostro programma verrebbe assegnato immediatamente ad un processore.

In ogni caso, constatiamo sperimentalmente che questo non succede. Ciò è sicuramente dovuto almeno in parte alla presenza di una sezione sequenziale nel codice e agli overhead di creazione e gestione dei vari threads usati.

A causa della grande disparità tra i risultati teorici attesi e quelli sperimentali immaginiamo che questo non fosse l'unico problema. Ne abbiamo una conferma anche dal fatto che aumentare il numero di step, e quindi il carico parallelo, non migliora la curva in Figure 2.

Guardando il codice, pensiamo che si possa ottimizzare facendo una reduction. Questa consente ad ogni thread di lavorare su una memoria privata, aggiornando quella condivisa solo al termine dell'esecuzione.

Utilizzando invece un array condiviso, ogni thread copia nella propria cache il blocco di memoria contenente l'intero array (o il sottoarray dove si trova la cella che vuole scrivere) sfruttando il principio di località spaziale. Ma, in questo modo, più thread lavora sullo stesso blocco di memoria, che viene quindi salvato sulla cache di ciascuno di essi. Ciò innesca dei meccanismi di protezione della coerenza delle cache che rallentano notevolmente il programma. In questo caso non mantenere la coerenza tra le cache non genererebbe errori perchè ogni thread scrive su una cella dell'array differente. Tuttavia, la macchina non è in grado di distinguere questa situazione dal caso generale.

Ora esaminiamo il caso in cui il numero di thread richiesti è maggiore di quelli effettivamente disponibili sulla macchina. Ci saremmo aspettati una saturazione delle prestazioni, a causa della mancanza delle risorse fisiche necessarie per eseguire i thread aggiuntivi richiesti. In ogni caso, abbiamo riscontrato un notevole miglioramento dei tempi di esecuzione.

Questo è probabilmente dovuto all'architettura del processore utilizzato per i test dal momento che prevede una tecnologia Intel HyperThreading, che permette di fare Simultaneous Multithreading, e quindi di evitare molti stalli della pipeline.

4 Domanda 4

Si può rendere il codice più efficiente? In base alla risposta che avete trovato per il punto 3, modificate il programma e ripetete gli esperimenti del punto 2 per dimostrare il miglioramento nel tempo di esecuzione.

4.1 Codice del programma

Come accennato nella Sezione 3, modifichiamo il codice in modo da utilizzare una variabile di reduction nel ciclo for.

```
// inclusione librerie  
#include <omp.h>  
#include <stdio.h>  
#include <math.h>
```

```

int num_threads = 4;
long long num_steps = 10000000;

int main()
{
    // set del numero di thread
    omp_set_num_threads(num_threads);

    // inizializzazione timer globale
    double start_time = omp_get_wtime();

    // dichiarazione variabili
    double step = 1.0/((double) num_steps);
    double sum=0.0;
    double pi;
    long long j; // indice in piu per il for finale

    // inizializzazione timer parallelo, con solo
    //    ↪ overhead di creazione thread
    double start_parallel_time = omp_get_wtime();

    #pragma omp parallel reduction(+:sum)
    {
        double x;
        #pragma omp for schedule(static)
        for (j=0;j<num_steps;j++) {
            x = (j+0.5)*step;
            sum = sum + 4.0/(1.0+x*x);
        }
    }

    pi=step*sum;

    // calcolo tempo totale e tempo parallelo
    double total_time = omp_get_wtime() - start_time;
    double total_parallel_time = omp_get_wtime() -
        ↪ start_parallel_time;

    // stampe finali
    printf(" Approssimazione di pi: -%.16f\nNumero di -
        ↪ threads: -%d\nNumero di passi: -%lld\nTempo -
        ↪ totale di esecuzione: -%.6f\n\nTempo totale -
        ↪ calcolo parallelo: -%.6f\n", pi, num_threads,
        ↪ num_steps, total_time, total_parallel_time);
}

```

```

    return 0;
}

```

4.2 Risultati dei test

Notiamo, come previsto, un miglioramento delle prestazioni del codice: osserviamo ora una scalabilità forte fino a 4 threads e discreti risultati fino a 8 threads. Osserviamo inoltre che le prestazioni del programma migliorano notevolmente per 10^9 step rispetto che per 10^7 step, in quanto aumenta il carico di lavoro in parallelo.

Ci saremmo aspettati invece che, per un numero di threads minore di quelli presenti nella macchina, lo speedup procedesse in modo pressochè lineare. Constatiamo invece sperimentalmente un deterioramento delle prestazioni probabilmente dovuto all'overhead della creazione e della gestione di un numero elevato di thread, a carico parallelizzabile fisso.

A differenza del caso precedente, quando il numero di threads supera quelli presenti fisicamente nella macchina, notiamo una saturazione delle prestazioni. Pensiamo che questo si possa spiegare ricordando che, mentre nel caso precedente il miglioramento era probabilmente dovuto al fatto che l'architettura HyperThreading usasse i thread in eccesso per nascondere i lunghi stalli della pipeline causati dall'accesso in memoria, in questa versione migliorata del programma i vari thread usano solo variabili private, lasciando al sistema un margine di ottimizzazione pressoché nullo.

Table 3: Risultati per 10^7 step

Threads	Tempo (s)	Speedup
1	0.0288	1.00
2	0.0146	1.97
4	0.00815	3.53
8	0.0058	4.97
12	0.0047	6.13
16	0.0044	6.55
32	0.00375	7.68

Table 4: Risultati per 10^9 step

Threads	Tempo (s)	Speedup
1	3.14385	1.00
2	1.5753	2.00
4	0.8261	3.81
8	0.53165	5.91
12	0.40965	7.67
16	0.32655	9.63
32	0.299	10.51

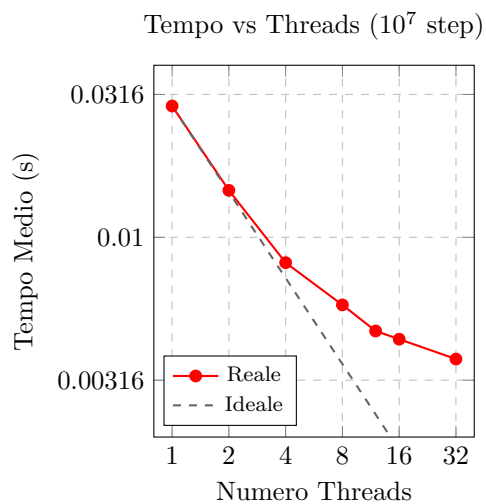


Figure 3: Tempo Medio vs Ideale per 10^7 step

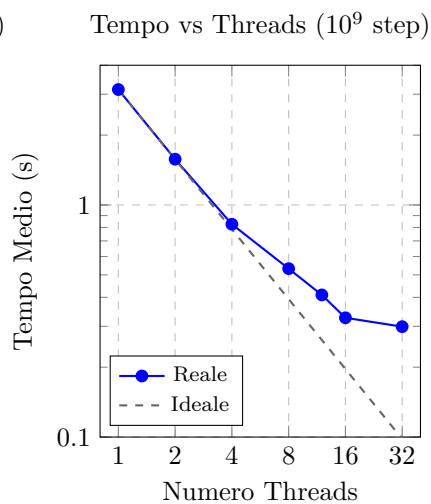


Figure 4: Tempo medio vs Ideale per 10^9 step